

Natural Language Processing (NLP)

99

Today

1. Define what NLP is and its importance in AI and everyday applications.
2. Explain and implement text preprocessing steps: **tokenization, stemming, lemmatization**.
3. Understand **Sentiment Analysis** as a core NLP task.

100

What is Natural Language Processing (NLP)?

-
- **Natural Language Processing (NLP)** is a subfield of **Artificial Intelligence (AI)** that enables machines to **understand, interpret, generate, and interact** using **human language**.
 - **Applications:**
 - Chatbots (e.g., Siri, Alexa)
 - Translation (Google Translate)
 - Sentiment Analysis (Product Reviews)
 - Speech-to-Text / Text-to-Speech

101

Goals of NLP

-
- Understand the **structure** and **meaning** of text or speech.
 - Convert **natural language** into a format computers can process.
 - Generate meaningful language in return.

102

Challenges in NLP

-
- Confusion
 - Context and Co-reference
 - Multilinguality
 - Misspelling

103

NLP Concepts

-
- Token: A single word (or unit)
 - Document: One complete response
 - Corpus: A collection of responses (all survey answers)

104

Text Cleaning and Preprocessing

-
- **Tokenization:** process of **splitting text into individual units**, usually words or punctuation marks.
 - **Example:**
 - **Raw Text:**
“The instructor was AMAZING and very clear!!”
 - **Tokenized:**
["The", "instructor", "was", "AMAZING", "and", "very", "clear", "!!"]
 - Break sentences into **tokens** to work with each word individually.

105

Text Cleaning and Preprocessing

-
- **Lowercasing:**
 - Tokenized, usually **normalize** text to remove irrelevant differences:
 - Convert everything to **lowercase**
 - Remove **punctuation** that doesn't add meaning
 - **Example:**
 - Tokenized:
["The", "instructor", "was", "AMAZING", "and", "very", "clear", "!!"]
 - After normalization:
["instructor", "amazing", "clear"]

106

Text Cleaning and Preprocessing

- **Stopword Removal**

- the, is, and, in, of, to, was
- These are **syntactic glue** they don't carry significant meaning in most analyses.
- Removing stopwords **reduces noise**, especially when building word frequencies or topic models.

107

Text Cleaning and Preprocessing

- **Stemming :**

- Removes suffixes
- Fast, but often **inaccurate**
- "teaching" → "teach"

- **Lemmatization (Smarter):**

- Uses vocabulary and grammar rules
- Converts to the **dictionary base form**
- "better" → "good"
- "studies" → "study"

108

Bag of Words (BoW)

- BoW is a **simple model** that:
 - Ignores grammar, word order, and context
 - Treats each document as a “bag” of words
 - Just counts **how often** each word appears
- It doesn't care **where** or **how** a word appears only **how many times**.

109

Bag of Words (BoW)

- Question: “What would you improve in this course?”
- Responses:
 - “More feedback on assignments.”
 - “Too slow sometimes.”
 - “Better materials and less repetition.”

Word	Frequency
feedback	1
assignments	1
slow	1
materials	1
repetition	1

110

Example:

1. "I love NLP"
 2. "I love AI and NLP"
- Vocabulary:
['love', 'NLP', 'AI']

Doc	I	love	NLP	AI	and
1	1	1	1	0	0
2	1	1	1	1	1

111

Sparse Matrix Representation

- Most documents use **only a few words** from the full vocabulary.
- For a dataset with:
 - 10,000 documents
 - 100,000 unique words
- BoW becomes a **huge matrix full of zeros** → called a **sparse matrix**.

112

Problem with BoW

- Common words like "the", "is", "and" have high counts but add little meaning.
- Solution: **TF-IDF**

113

Example

- `from sklearn.feature_extraction.text import CountVectorizer`
- `corpus = [ "I love NLP",`
- `"I love AI and NLP " ]`
- `vectorizer = CountVectorizer()`
- `X = vectorizer.fit_transform(corpus)`
- `print(vectorizer.get_feature_names_out())`
- `print(X.toarray())`

Document	ai	and	love	nlp
1: "I love NLP"	0	0	1	1
2: "I love AI and NLP"	1	1	1	1

114

Example Using TF-IDF

-
- `from sklearn.feature_extraction.text import TfidfVectorizer`
 - `vectorizer = TfidfVectorizer()`
 - `X = vectorizer.fit_transform(corpus)`
 - `print(vectorizer.get_feature_names_out())`
 - `print(X.toarray())`
 - `corpus = [ "I love NLP",`
 - `"I love AI and NLP"]`

Document	ai	and	love	nlp
"I love NLP"	0.00	0.00	0.7071	0.7071
"I love AI and NLP"	0.50	0.50	0.5000	0.5000

115

What is Sentiment Analysis?

-
- **Sentiment analysis** is a natural language processing task that determines the **emotional tone** behind a piece of text.
 - It answers questions like:
 - Is this review **positive**, **negative**, or **neutral**?
 - Does this tweet express **anger**, **joy**, or **sadness**?
 - Is the customer **satisfied** or **unhappy**?

116

Types of Sentiment Analysis

Type	Description	Example
Binary Classification	Positive vs. Negative	"I love this phone!"
Multi-class Classification	Positive, Neutral, Negative	"The product is okay."
Emotion Detection	Joy, anger, sadness, surprise, etc.	"I'm so frustrated with the delay!"

117

How Does Sentiment Analysis Work?

-
- Rule-Based Approaches:
 - These use dictionaries of words with assigned sentiment scores.
 - Example:
 - `from textblob import TextBlob`
 - `TextBlob("This movie was amazing!").sentiment`
 - `Sentiment(polarity=0.75, subjectivity=0.6)`

118

How Does Sentiment Analysis Work?

-
- Rule-Based Approaches
 - Machine Learning Approaches:
 - Collect labeled data (e.g., IMDb movie reviews)
 - Preprocess (clean, tokenize, remove stop words)
 - Convert to vectors (BoW, TF-IDF)
 - Train a classifier:
 - Naive Bayes
 - Logistic Regression
 - SVM
 - Evaluate (accuracy, precision, recall)

119

Case Study: Sentiment Analysis on Amazon Product Reviews

-
- **Goal:**
 - Classify product reviews as **positive** or **negative**.
 - **Dataset:**
 - Amazon reviews dataset (CSV)
 - **Preprocessing**

120

Step 1: Import Required Libraries

- import pandas as pd
- df = pd.read_csv('amazon_reviews.csv')
- print(df.head())

review_text	label
"I love this product!"	1
"Not what I expected. Very disappointing"	0
"Works well, good value."	1
"Stopped working after a week."	0
"Excellent! Highly recommended."	1

121

Step 2: Preprocess the Text

- import nltk
- from nltk.corpus import stopwords
- from nltk.stem import PorterStemmer
- import re
- nltk.download('stopwords')
- stop_words = set(stopwords.words('english'))
- stemmer = PorterStemmer()
- def preprocess_text(text):
- text = re.sub(r'^\w\s', '', text.lower())
- tokens = text.split()
- tokens = [stemmer.stem(word) for word in tokens if word not in stop_words]
- return ' '.join(tokens)
- df['cleaned_review'] = df['review_text'].astype(str).apply(preprocess_text)

Original: "I loved this product! It's amazing and works well."

Cleaned: "love product amaz work well"

122

Step 3: Vectorize the Text

-
- `from sklearn.feature_extraction.text import CountVectorizer`
 - `vectorizer = CountVectorizer(max_features=5000)`
 - `X = vectorizer.fit_transform(df['cleaned_review']).toarray()`
 - `y = df['label']`

123

Step 4: Split the Data

-
- `from sklearn.model_selection import train_test_split`
 - `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`

124

Step 5: Train Using Bagging with Random Forest

-
- `from sklearn.ensemble import BaggingClassifier, RandomForestClassifier`
 - `base_rf = RandomForestClassifier(n_estimators=100, max_depth=None, random_state=42)`
 - `bagging_model = BaggingClassifier(base_estimator=base_rf, n_estimators=10, random_state=42)`
 - `bagging_model.fit(X_train, y_train)`

125

Step 6: Evaluate the Model

-
- `from sklearn.metrics import accuracy_score, classification_report, confusion_matrix`
 - `y_pred = bagging_model.predict(X_test)`
 - `print("Accuracy:", accuracy_score(y_test, y_pred))`
 - `print("Classification Report:\n", classification_report(y_test, y_pred))`

126